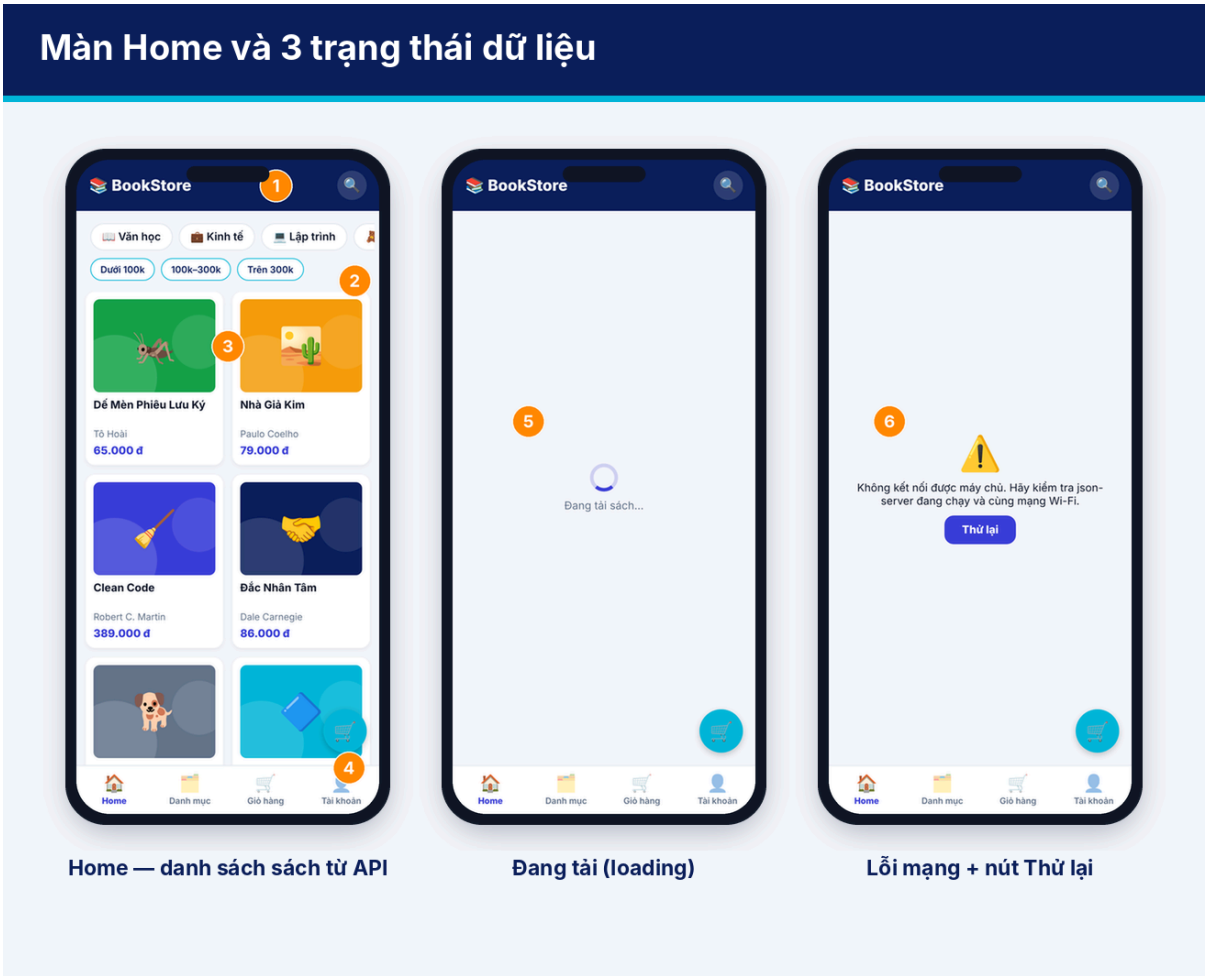
THỰC HÀNH REACT NATIVE · BUỔI 7 · NAVIGATION, REDUX & API

Buổi 7 — BookStore: Điều hướng, Redux, API & Tìm kiếm

Mục tiêu

- Dựng đúng giao diện **BookStore** (Home, Chi tiết, Giỏ hàng, Thanh toán, Danh mục, Tài khoản) bằng Expo, React Native và TypeScript.
- Thành thạo **Bottom Tab + Native Stack**, biết truyền tham số giữa các màn và tự dựng tab bar.
- Dùng **Redux Toolkit** cho giỏ hàng và tài khoản, bọc lại bằng hook useCart / useAuth để giao diện không dính trực tiếp vào Redux.
- Gọi **API** bằng Axios + json-server, xử lý đủ loading / lỗi / thử lại; làm tìm kiếm có **debounce** và lọc theo nhiều điều kiện.

Bài	Nội dung	Kỹ năng chính
Bài 7.1	Điều hướng 2 tầng + Tab Bar tự dựng	Stack, Bottom Tab, tabBar tuỳ biến
Bài 7.2	Truyền tham số giữa các màn	navigate(name, params), route.params
Bài 7.3	Giỏ hàng bằng Redux Toolkit	createSlice, useMemo, adapter useCart
Bài 7.4	Tài khoản đổi theo trạng thái	authSlice, isLoggedIn, render có điều kiện
Bài 7.5	Gọi API sách: loading, lỗi, thử lại	Axios, services, Promise.all, useEffect
Bài 7.6	Gọi API danh mục	Dùng lại LoadingView / ErrorView
Bài 7.7	Tìm kiếm có debounce, không dấu	useDebounce, setTimeout / clearTimeout
Bài 7.8	Lọc kết hợp + liên kết tab Danh mục	filterBooks, navigation.setParams

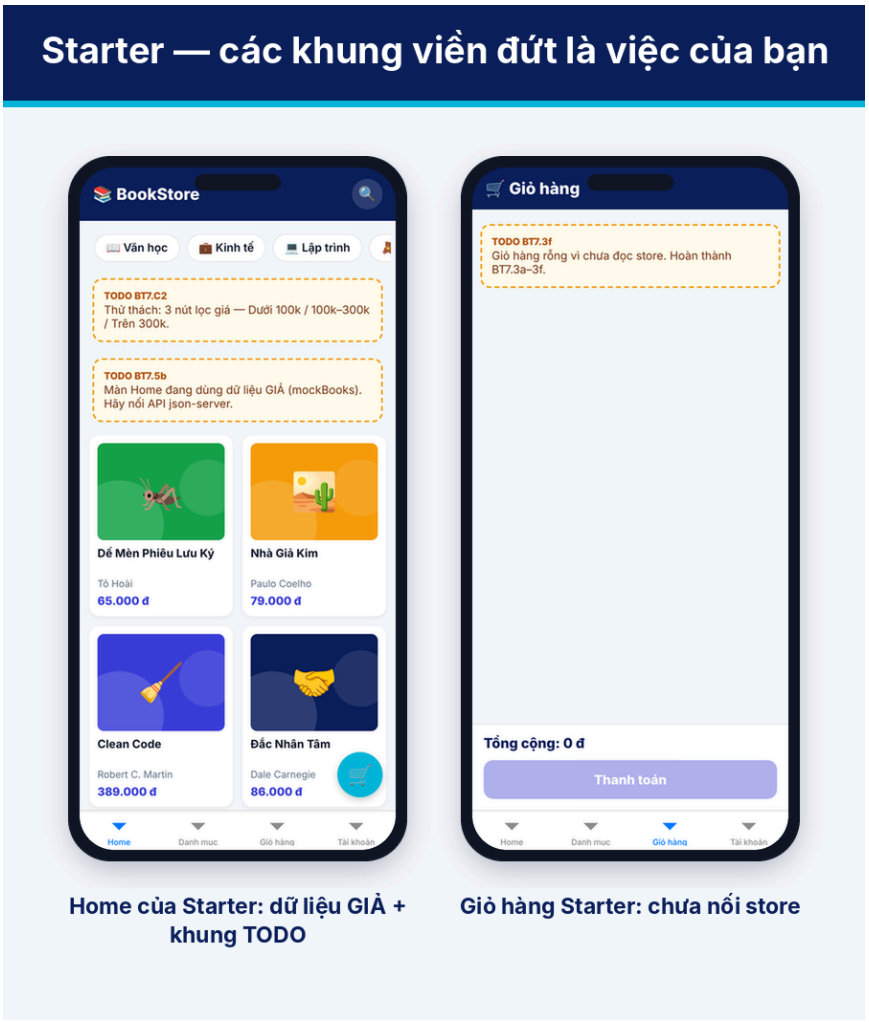


Hình 1. Giao diện đích của Home: có dữ liệu, đang tải, lỗi mạng.

1. Chuẩn bị

```
# giải nén BookStore_Buoi7_Starter.zip rồi mở thư mục bằng VS Code (cần Node.js 20+)
cd BookStore_Buoi7_Starter
npm install
npm run api          # Terminal 1: json-server chạy ở http://localhost:3000
npx expo start       # Terminal 2: quét QR bằng Expo Go; nhấn w để mở bản web
npx tsc --noEmit     # kiểm tra kiểu — chạy trước khi nộp bài
```

- **Tìm việc cần làm:** Ctrl+Shift+F, tìm chuỗi `TODO BT7`. Mỗi chỗ có dạng `// TODO BT7.3a: ...`.
- **Khung viền đứt màu cam** trên màn hình là chỗ chưa làm; làm xong thì xoá thẻ `<TodoBox ... />` tương ứng.
- **Điện thoại thật:** máy tính và điện thoại cùng Wi-Fi; nếu không gọi được API, chạy `EXPO_PUBLIC_API_URL=http://<IP-máy-tính>:3000 npx expo start -c`.
- **Tài khoản demo:** demo@bookstore.vn / 123456. Đăng nhập trong buổi này là **giả lập** (nút "Đăng nhập thử").



Hình 2. Starter khi mới chạy: dữ liệu giả và các khung TODO cam. Làm xong 8 bài thì màn hình sẽ giống Hình 1.

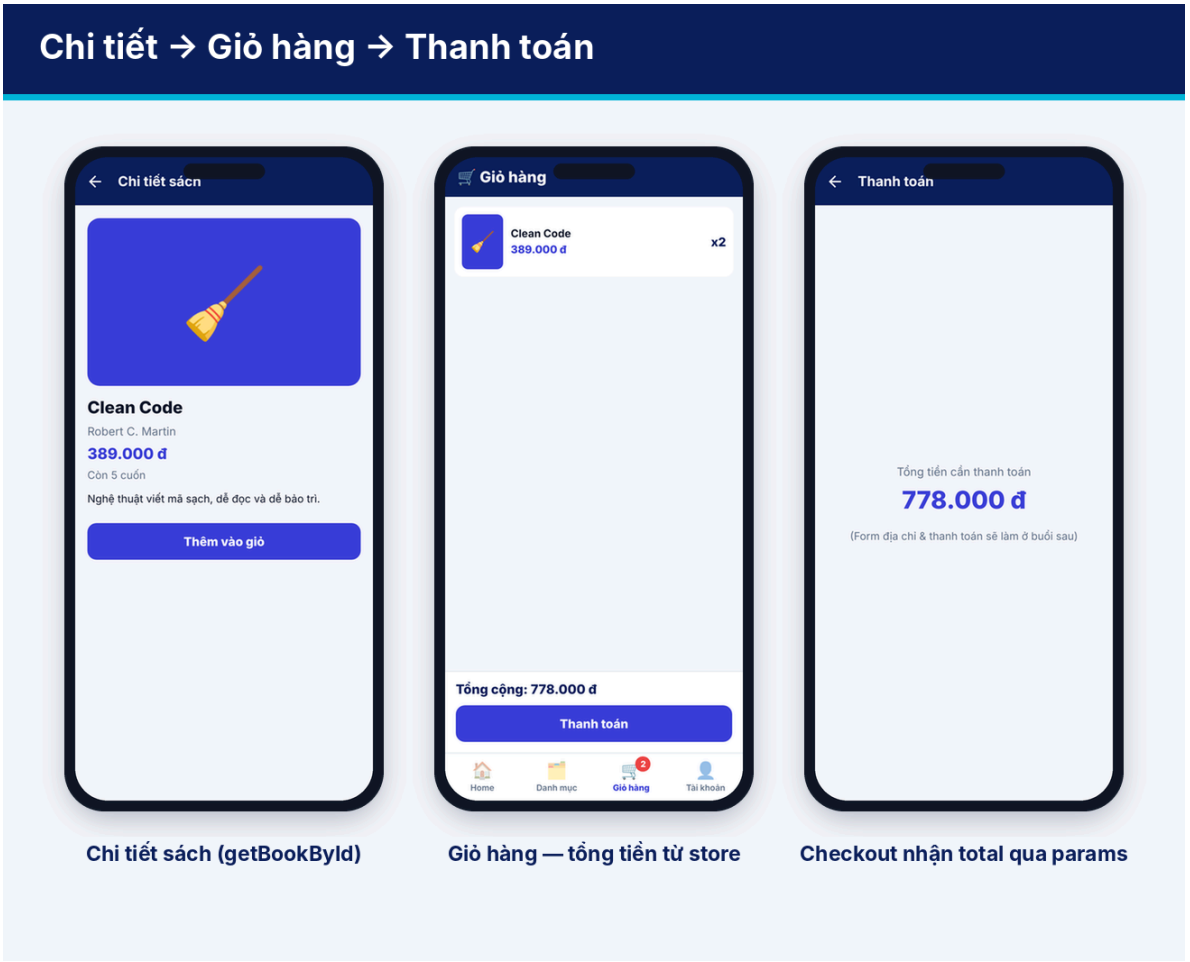
Cấu trúc thư mục

```
BookStore_Buoi7_Starter/
├── App.tsx              # Provider (Redux) + NavigationContainer + RootNavigator
├── db.json              # dữ liệu cho json-server: books ×12, categories ×5, users…
└── src/
    ├── api/             # client.ts (axios + interceptor), config.ts   ← có sẵn
    ├── components/      # TabBar, BookCard, BookGrid, SearchBar, EmptyState… ← có sẵn
    ├── navigation/      # types.ts, RootNavigator, TabNavigator, CustomTabBar ← Bài 7.1
    ├── screens/         # Home, BookDetail, Cart, Checkout, Categories, Account
    ├── services/        # bookService, categoryService                ← Bài 7.5, 7.6
    ├── store/           # index, cartSlice, authSlice, hooks          ← Bài 7.3, 7.4
    ├── hooks/           # useDebounce                                 ← Bài 7.7
    └── utils/           # filters.ts (normalize, filterBooks)         ← Bài 7.7, 7.8
```

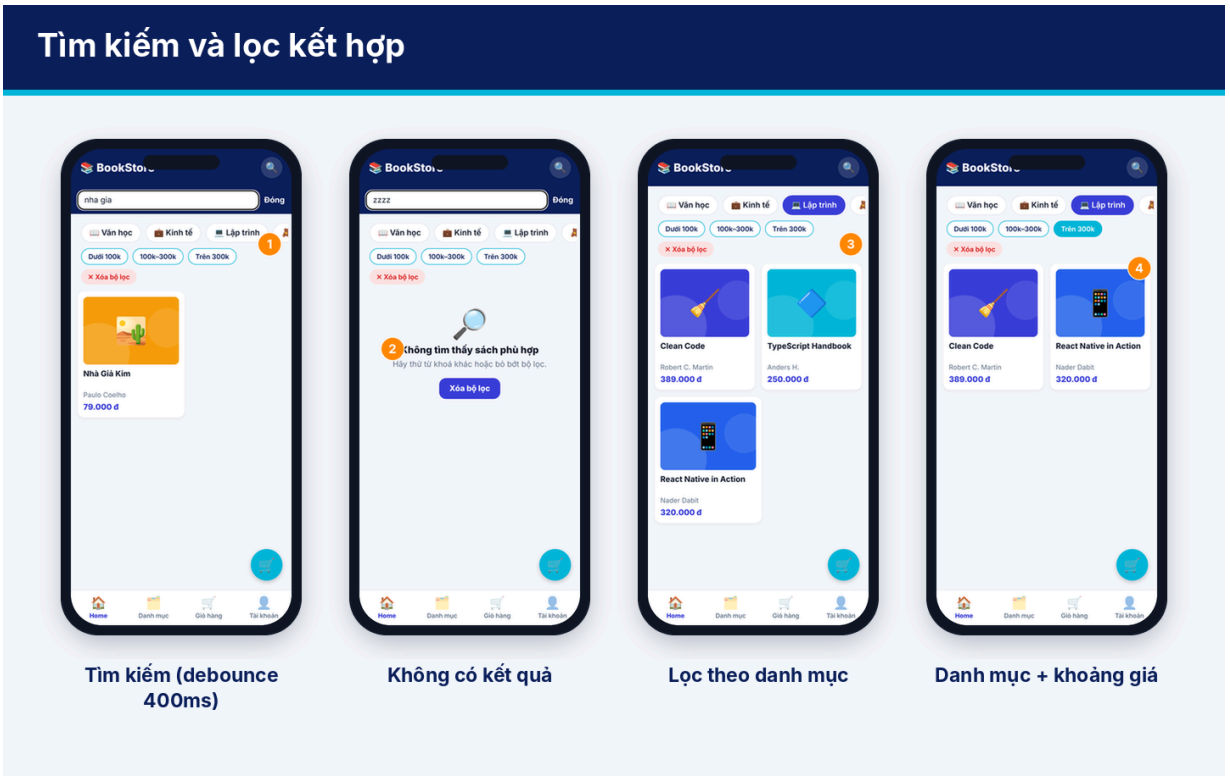
> **Lưu ý:** Starter chạy được ngay vì các màn đang dùng dữ liệu giả. Làm lần lượt từ Bài 7.1 đến 7.8, xong mỗi bài mở app xem kết quả rồi hãy qua bài sau.

2. Giao diện đích và code dùng chung

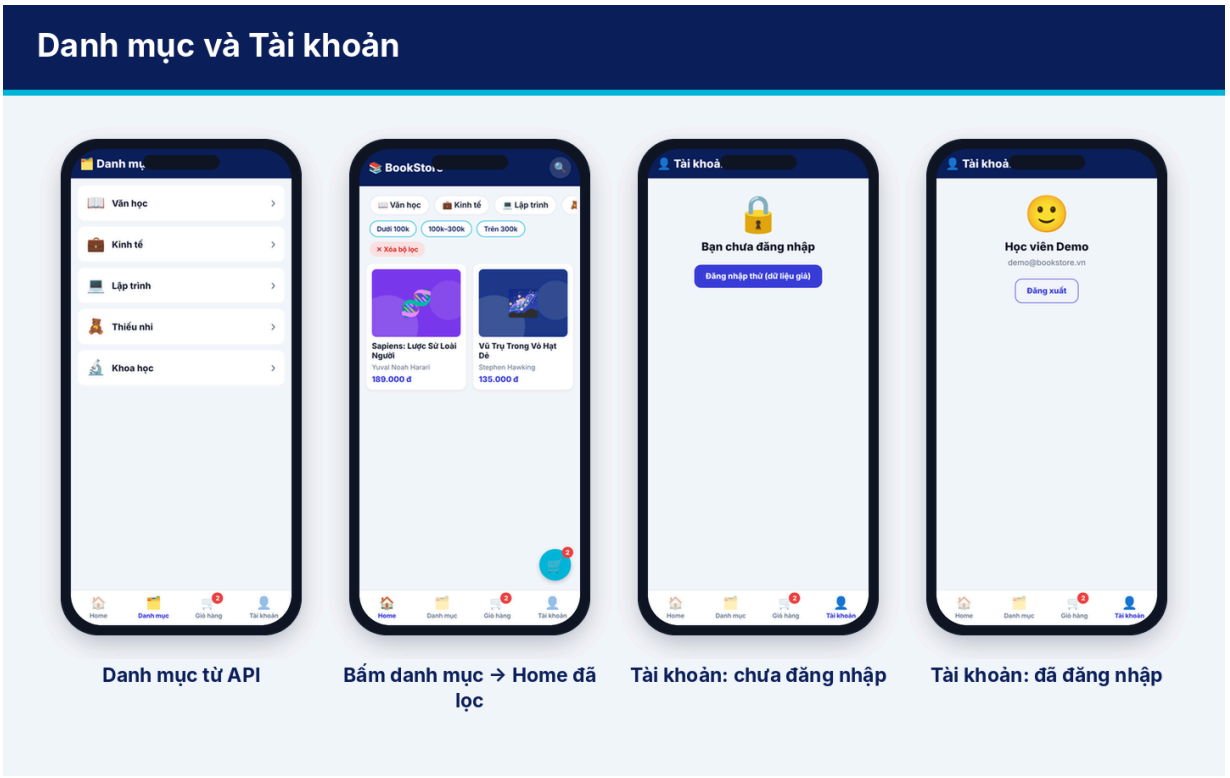
Màu dùng trong app: navy #0B1F5C cho header, indigo #3B3FD8 cho nút chính và giá, cyan #00B8D9 cho nút giỏ nổi, nền #F4F6FB, thẻ trắng bo góc 16. Mô tả từng màn xem trong docs/UI.md, ảnh gốc ở docs/ui/.



Hình 3. Chi tiết sách → Giỏ hàng (badge đỏ trên tab) → Thanh toán.



Hình 4. Tìm kiếm có debounce, không có kết quả, lọc theo danh mục và khoảng giá.



Hình 5. Danh mục từ API và hai trạng thái của tab Tài khoản.

Mấy file dưới đây project đã có sẵn, bạn chỉ cần đọc cho hiểu, không phải chép lại.

📄 **src/types.ts**

```
export interface Book {
  id: number; title: string; author: string; price: number;
  categoryId: number; stock: number; description: string;
  emoji: string; color: string;   // màu nền "bìa sách"
}
export interface Category { id: number; name: string; emoji: string; }
export interface CartItem {
  bookId: number; title: string; price: number;
  emoji: string; color: string; stock: number; quantity: number;
}
export interface User { id: number; name: string; email: string; phone?: string; }
```

📄 **src/navigation/types.ts (rút gọn)**

```
import type { NavigatorScreenParams } from "@react-navigation/native";

export type TabParamList = {
  Home: { categoryId?: number } | undefined;
  Categories: undefined;
  Cart: undefined;
  Account: undefined;
};
export type RootStackParamList = {
  Tabs: NavigatorScreenParams<TabParamList> | undefined;
  BookDetail: { bookId: number };
  Checkout: { total: number };
};
// → navigate("BookDetail", { bookId: 3 }) sai tên màn hoặc thiếu params sẽ bị TypeScript báo lỗi
```

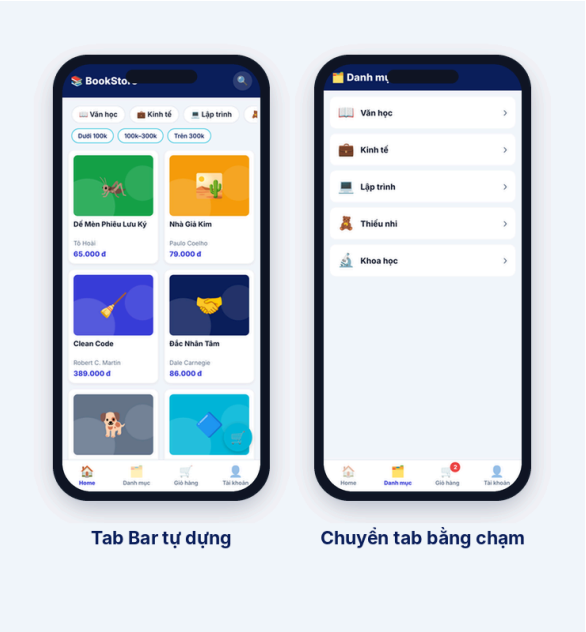
📄 **src/api/client.ts (rút gọn)**

```
export const api = axios.create({ baseURL: API_URL, timeout: 8000 });   // MỘT instance dùng
chung

api.interceptors.request.use((config) => {          // tự gắn token (nếu có)
  const token = tokenProvider();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
api.interceptors.response.use((res) => res, (error) => {   // gom lỗi về 1 câu tiếng Việt
  let message = "Đã có lỗi xảy ra. Vui lòng thử lại.";
  if (axios.isAxiosError(error)) {
    if (!error.response) message = "Không kết nối được máy chủ. Hãy kiểm tra json-server…";
    else if (error.response.status === 404) message = "Không tìm thấy dữ liệu.";
    else if (error.response.status >= 500) message = "Máy chủ đang gặp sự cố.";
  }
  return Promise.reject(new Error(message));
});
export const getErrorMessage = (e: unknown) => (e instanceof Error ? e.message : "Đã có lỗi xảy
ra.");
```

Bài 7.1: Điều hướng 2 tầng và Tab Bar tự dựng

Yêu cầu: app có một **Native Stack** ở ngoài cùng (Tabs → BookDetail → Checkout) và một **Bottom Tab** nằm trong màn Tabs (Home, Danh mục, Giỏ hàng, Tài khoản). Thanh tab phải là bản tự dựng, không dùng thanh mặc định của thư viện.



Hình 6. Trái: Tab Bar tự dựng cần đạt. Phải: thanh mặc định của thư viện (đang có ở Starter).

Lý thuyết nhanh

```
RootStack (Native Stack)
  ├─ Tabs ——— Bottom Tab:  Home | Categories | Cart | Account     ← tabBar={CustomTabBar}
  ├─ BookDetail   (nằm ở Stack → che lên cả Tab Bar, có nút ← tự động)
  └─ Checkout

// Bottom Tab cho phép đưa vào một component riêng qua prop tabBar.
// Component đó nhận { state, descriptors, navigation } – ta đổi thành mảng TabItem cho TabBar vẽ.
```

Bước 1 — Đọc component TabBar (thuần giao diện)

A. CODE CÓ SẴN — chép vào project (phần có sẵn)

📄 **src/components/TabBar.tsx (rút gọn)**

```
export interface TabItem { key: string; label: string; emoji: string; badge?: number; }
interface Props { items: TabItem[]; activeKey: string; onPress: (key: string) => void; }

// TabBar chưa biết gì về thư viện điều hướng – chỉ vẽ items và báo onPress(key)
export function TabBar({ items, activeKey, onPress }: Props) { … }
```

Bước 2 — Viết CustomTabBar (cầu nối)

2.1 CustomTabBar

Lấy state.routes biến thành mảng items. Khi bấm tab thì phát sự kiện tabPress, sau đó chuyển màn nếu tab đó chưa đang mở. Nhớ xoá dòng void … và thẻ TodoBox cũ trong file.

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/navigation/CustomTabBar.tsx** TODO

```
import type { BottomTabBarProps } from "@react-navigation/bottom-tabs";
import { TabBar, TabItem } from "../components/TabBar";

const EMOJI: Record<string, string> = { Home: "🏠", Categories: "🗂️", Cart: "🛒", Account: "👤"
};

export function CustomTabBar({ state, descriptors, navigation }: BottomTabBarProps) {
  // TODO BT7.1a: items: TabItem[] = state.routes.map(...)  → { key, label, emoji }
  //   label = descriptors[route.key].options.title ?? route.name ; emoji = EMOJI[route.name]
  // TODO BT7.1a: onPress(key): tìm route; navigation.emit({ type: "tabPress", target: route.key,
canPreventDefault: true })
  //   nếu tab chưa focus và chưa bị preventDefault → navigation.navigate(route.name)
  // TODO BT7.1a: return <TabBar items={items} activeKey={state.routes[state.index].key}
onPress={onPress} />
}
```

2.2 Gắn vào TabNavigator

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/navigation/TabNavigator.tsx** TODO

```
<Tab.Navigator
  screenOptions={{ headerShown: false }}
  // TODO BT7.1b: thêm prop  tabBar={(props) => <CustomTabBar {...props} />}
>
  <Tab.Screen name="Home" component={HomeScreen} options={{ title: "Home" }} />
  <Tab.Screen name="Categories" component={CategoriesScreen} options={{ title: "Danh mục" }} />
  …
</Tab.Navigator>
```

Bước 3 — Đăng ký màn BookDetail trong Stack

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/navigation/RootNavigator.tsx** TODO

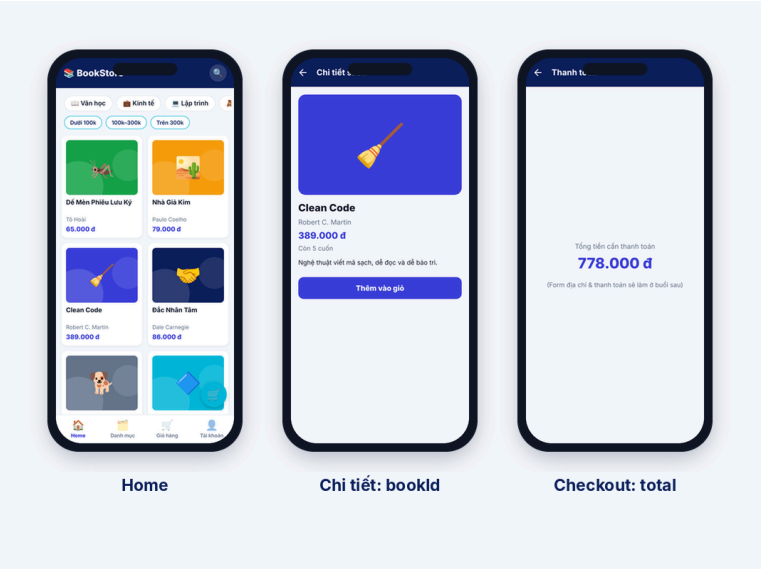
```
<Stack.Navigator screenOptions={stackScreenOptions}>
  <Stack.Screen name="Tabs" component={TabNavigator} options={{ headerShown: false }} />
  {/* TODO BT7.1c: <Stack.Screen name="BookDetail" component={BookDetailScreen} options={{ title:
"Chi tiết sách" }} /> */}
  <Stack.Screen name="Checkout" component={CheckoutScreen} options={{ title: "Thanh toán" }} />
</Stack.Navigator>
```

Cách tự kiểm tra:

- Thanh tab đúng như Hình 6 (bên trái): emoji + nhãn, mục đang chọn màu indigo.
- Bấm từng tab chuyển đúng màn; bấm lại tab đang mở không gây lỗi.
- Thử gõ sai tên màn trong navigate: TypeScript phải báo lỗi đỏ. Xem xong nhớ sửa lại.
- Chạy npx tsc --noEmit, không được có lỗi.

Bài 7.2: Truyền tham số giữa các màn

Yêu cầu: bấm thẻ sách ở Home → mở đúng sách đó; bấm "Thanh toán" ở Giỏ → Thanh toán hiển thị đúng tổng tiền.



Hình 7. Home → Chi tiết gửi bookId; Giỏ → Thanh toán gửi total.

Lý thuyết nhanh

```
navigation.navigate("BookDetail", { bookId: 3 });   // gửi
const { bookId } = route.params;                      // nhận (đã có kiểu nhờ
RootStackParamList)
```

Bước 1 — Phía nhận (đã viết sẵn)

A. CODE CÓ SẴN — chép vào project (phần có sẵn)

📄 **BookDetailScreen.tsx · CheckoutScreen.tsx (rút gọn)**

```
export function BookDetailScreen({ route }: RootScreenProps<"BookDetail">) {
  const { bookId } = route.params;          // ← nhận bookId
  …
}
// CheckoutScreen:
<Text style={styles.total}>{formatPrice(route.params.total)}</Text>   // ← nhận total
```

Bước 2 — Phía gửi

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/screens/HomeScreen.tsx** TODO

```
<BookGrid
  data={visibleBooks}
  // TODO BT7.2a: onPressBook → navigation.navigate("BookDetail", { bookId: b.id })
  onPressBook={() => {}}
/>
```

📄 **src/screens/CartScreen.tsx** TODO

```
<Pressable
  disabled={items.length === 0}
  // TODO BT7.2b: navigation.navigate("Checkout", { total: totalPrice })
  onPress={() => void navigation}
>
```

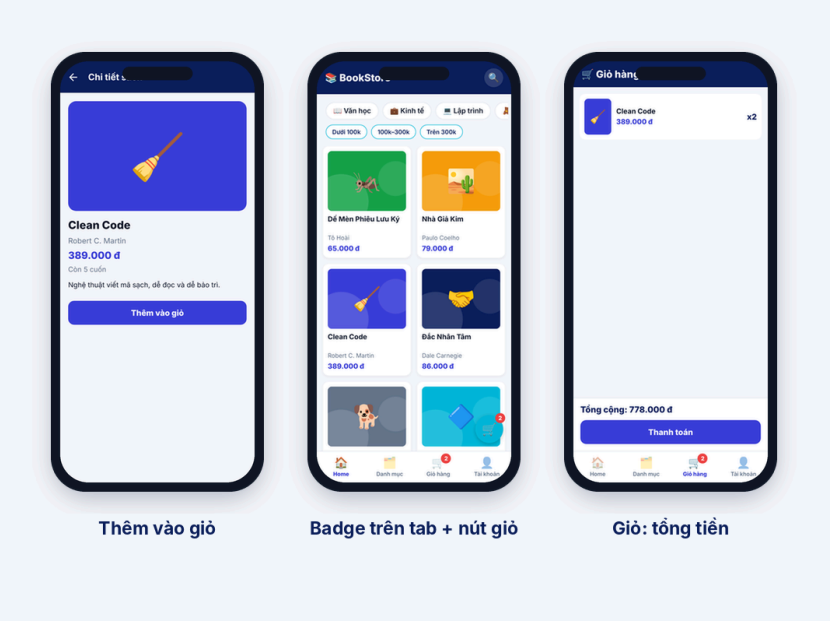
Cách tự kiểm tra:

- Bấm 3 cuốn sách khác nhau → mỗi lần hiện đúng tên, giá của sách đó.
- Nút ← ở Chi tiết quay về Home; vị trí cuộn được giữ.

- Ý này chỉ kiểm được sau khi xong Bài 7.3 (lúc đó giỏ mới có hàng): bấm Thanh toán phải ra đúng tổng tiền.

Bài 7.3: Giỏ hàng bằng Redux Toolkit

Yêu cầu: thêm sách vào giỏ từ màn Chi tiết; cùng một sách thêm nhiều lần thì tăng số lượng nhưng không vượt tồn kho (stock). Badge ở tab Giỏ hàng và nút giỏ nổi hiện tổng số lượng; màn Giỏ hiện tổng tiền.



Hình 8. Thêm vào giỏ → badge trên tab → Giỏ hàng có tổng tiền.

Lý thuyết nhanh

```
createSlice({ name, initialState, reducers })   // gom state + reducers + action creators
// Trong reducer được viết "như mutate": state.items.push(x) — Immer tạo state mới bất biến phía sau.
useAppSelector((s) => s.cart.items)              // đọc state; component render lại khi phần này đổi
useMemo(() => items.reduce(...), [items])        // tính tổng chỉ khi items đổi
// ADAPTER: UI chỉ gọi useCart() — sau này đổi Redux sang Zustand chỉ sửa hooks.ts, UI giữ nguyên.
```

Bước 1 — Store (đã có sẵn)

A. CODE CÓ SẴN — chép vào project (phần có sẵn)

📄 **src/store/index.ts · App.tsx (rút gọn)**

```
export const store = configureStore({ reducer: { cart, auth, wishlist } });
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

// App.tsx:  <Provider store={store}> … </Provider>
```

Bước 2 — Viết reducer cho cartSlice

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/store/cartSlice.ts** TODO

```
const clamp = (q: number, stock: number) => Math.max(1, Math.min(q, stock));   // có sẵn

reducers: {
  // TODO BT7.3a: addItem — đã có bookId → tăng quantity (clamp ≤ stock); chưa có → push({
...payload, quantity: 1 })
  addItem(state, action: PayloadAction<Omit<CartItem, "quantity">>) { … },
  // TODO BT7.3a: removeItem — lọc bỏ item có bookId === action.payload
  removeItem(state, action: PayloadAction<number>) { … },
  // TODO BT7.3a: setQuantity — tìm item theo bookId, gán quantity = clamp(payload.quantity,
item.stock)
  setQuantity(state, action: PayloadAction<{ bookId: number; quantity: number }>) { … },
  // TODO BT7.3a: clearCart — state.items = []
  clearCart(state) { … },
}
```

Bước 3 — Tính tổng trong hook useCart

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/store/hooks.ts** TODO

```
export function useCart() {
  const dispatch = useAppDispatch();
  const items = useAppSelector((s) => s.cart.items);

  // TODO BT7.3b: totalQuantity = Σ quantity · totalPrice = Σ price × quantity  (reduce + useMemo
theo [items])
  const totalQuantity = useMemo(() => 0 * items.length, [items]);
  const totalPrice = useMemo(() => 0 * items.length, [items]);

  return { items, totalQuantity, totalPrice, add: …, remove: …, setQty: …, clear: … };
}
```

Bước 4 — Nối giao diện vào store

B. VIỆC CẦN LÀM — tự code thêm

📄 **BookDetail · Home · CustomTabBar · CartScreen** TODO

```
// TODO BT7.3c  BookDetailScreen: nút "Thêm vào giỏ"
<Pressable onPress={() => add({ bookId: book.id, title: …, price: …, emoji: …, color: …, stock:
… })}>

// TODO BT7.3d  HomeScreen: const { totalQuantity } = useCart();  → <FloatingCartButton
count={totalQuantity} … />

// TODO BT7.3e  CustomTabBar: badge = totalQuantity (từ useCart) khi route.name === "Cart"

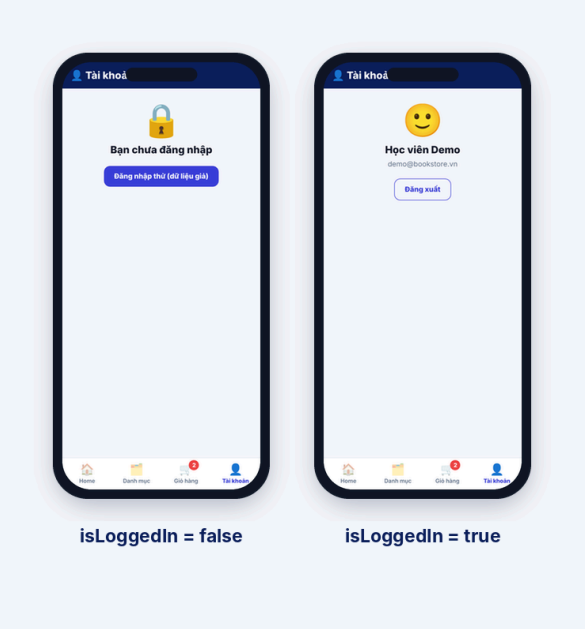
// TODO BT7.3f  CartScreen: const { items, totalPrice } = useCart();  rồi XOÁ <TodoBox
id="BT7.3f"> trong ListEmptyComponent
```

Cách tự kiểm tra:

- Thêm “Clean Code” 2 lần (stock 5) và một sách 389.000 đ → tổng tiền khớp phép tính tay.
- Cố thêm quá tồn kho: số lượng dừng ở mức stock, không tăng nữa.
- Badge trên tab Giỏ và nút giỏ nổi đều hiện tổng số lượng. Chuyển qua lại giữa các tab, giỏ không bị mất.

Bài 7.4: Tài khoản — giao diện đổi theo trạng thái

Yêu cầu: tab Tài khoản có 2 nhánh: chưa đăng nhập (🔒 + nút "Đăng nhập thử") và đã đăng nhập (tên, email, nút Đăng xuất). Trạng thái lấy từ Redux.



Hình 9. Hai trạng thái của tab Tài khoản.

Lý thuyết nhanh

```
isLoggedIn = !!token && !!user     // đăng nhập khi có CẢ token và user
{isLoggedIn ? <NhanhDaDangNhap /> : <NhanhChuaDangNhap />}   // render có điều kiện
```

Bước 1 — Reducer của authSlice

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/store/authSlice.ts** TODO

```
reducers: {
  // TODO BT7.4a: setCredentials – gán state.user = action.payload.user ; state.token =
action.payload.token
  setCredentials(state, action: PayloadAction<{ user: User; token: string }>) { … },
  updateUser(state, action) { … },   // có sẵn
  // TODO BT7.4a: logout – đưa user và token về null
  logout(state) { … },
}
```

Bước 2 — Hook useAuth và AccountScreen

B. VIỆC CẦN LÀM — tự code thêm

📄 **hooks.ts · AccountScreen.tsx** TODO

```
// hooks.ts
isLoggedIn: false,   // TODO BT7.4b: !!token && !!user

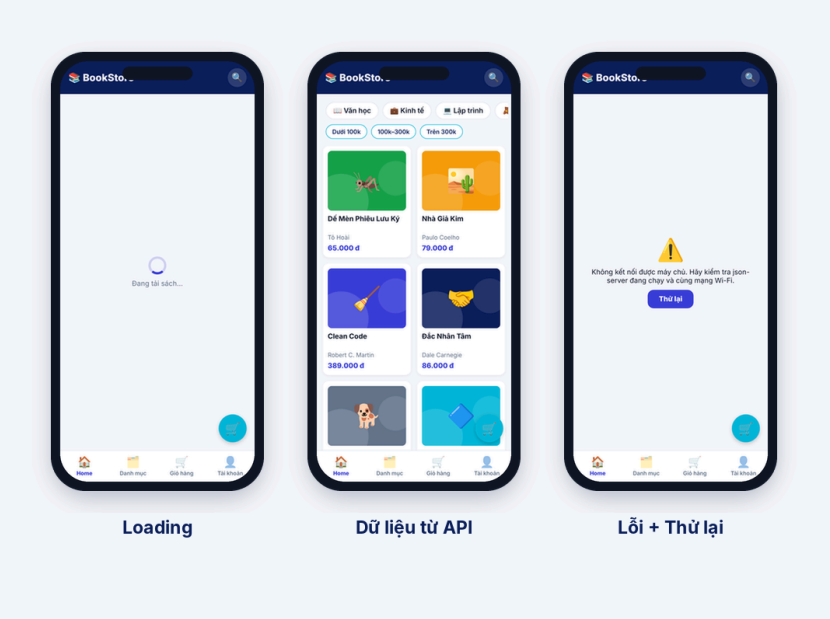
// AccountScreen.tsx
// TODO BT7.4d: nút đăng nhập thử → signIn({ id: 1, name: "Học viên Demo", email:
"demo@bookstore.vn" }, "fake-token")
//              nút Đăng xuất → signOut
```

Cách tự kiểm tra:

- Bấm Đăng nhập thử thì hiện tên và email, bấm Đăng xuất thì quay lại màn 🔒. Làm vài lần liền, giao diện phải đổi ngay.
- Reload app thì trạng thái về chưa đăng nhập (phần lưu lại chưa làm ở buổi này).

Bài 7.5: Gọi API sách — loading, lỗi, thử lại

Yêu cầu: Home gọi `GET /books` và `GET /categories` song song; có 3 trạng thái: đang tải, lỗi (kèm nút Thử lại), có dữ liệu. Màn Chi tiết gọi `GET /books/:id`.



Hình 10. Có dữ liệu · đang tải · lỗi mạng với nút Thử lại.

Lý thuyết nhanh

```
json-server biến db.json thành REST API:
  GET /books          GET /books/3          GET /books?categoryId=1          GET /categories

// Mẫu chuẩn cho mọi màn gọi API:
setLoading(true); setError(null);
try { const data = await getX(); setData(data); }
catch (e) { setError(getErrorMessage(e)); }
finally { setLoading(false); }

// Hai API độc lập → chạy SONG SONG:  const [b, c] = await Promise.all([getBooks(),
getCategories()]);
```

Bước 1 — Bật json-server và thử API

```
npm run api
# mở trình duyệt: http://localhost:3000/books      (phải thấy 12 cuốn sách dạng JSON)
#                 http://localhost:3000/books/3
#                 http://localhost:3000/categories
```

Bước 2 — Viết tầng services

Quy tắc: component không gọi axios trực tiếp; mọi lời gọi API nằm trong thư mục services và trả về dữ liệu đã có kiểu.

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/services/bookService.ts** TODO

```
export async function getBooks(params?: BookQuery): Promise<Book[]> {
  // TODO BT7.5a: const res = await api.get<Book[]>("/books", { params }); return res.data;
}
export async function getBookById(id: number): Promise<Book> {
  // TODO BT7.5a: api.get<Book>(`/books/${id}`) → trả res.data
}
```

Bước 3 — HomeScreen: state + load() + 3 trạng thái

A. CODE CÓ SẴN — LoadingView và ErrorView

📄 **src/components/ (cách dùng)**

```
<LoadingView label="Đang tải sách..." />
<ErrorView message={error} onRetry={load} />     // nút "Thử lại" gọi onRetry
```

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/screens/HomeScreen.tsx** TODO

```
export function HomeScreen({ navigation }: TabScreenProps<"Home">) {
  // TODO BT7.5b: state books, categories, loading, error (useState) rồi viết
  //   const load = useCallback(async () => { setLoading(true); setError(null);
  //     try { const [b, c] = await Promise.all([getBooks(), getCategories()]); ... }
  //     catch (e) { setError(getErrorMessage(e)); } finally { setLoading(false); } }, []);
  //   useEffect(() => { load(); }, [load]);
  const books = MOCK_BOOKS;            // ← dữ liệu giả, sẽ thay bằng state
  const categories = MOCK_CATEGORIES;

  // TODO BT7.5c: loading → <LoadingView/> · error → <ErrorView message={error} onRetry={load}/>
· còn lại → BookGrid
  ...
}
```

Bước 4 — BookDetailScreen

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/screens/BookDetailScreen.tsx** TODO

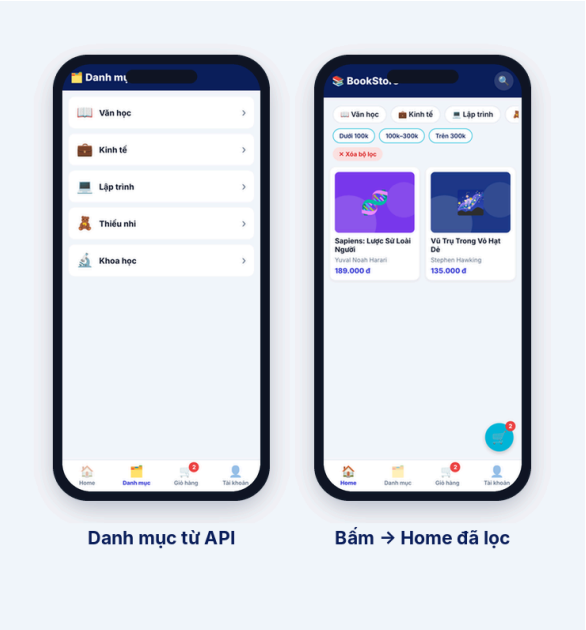
```
// TODO BT7.5d: state book / loading / error + load() gọi getBookById(bookId); chạy trong
useEffect(..., [bookId])
// TODO BT7.5e: if (loading) return <LoadingView/>;
//              if (error || !book) return <ErrorView message={error ?? "Không tìm thấy sách."}
onRetry={load}/>
```

Cách tự kiểm tra:

- Tắt json-server (Ctrl+C) rồi mở app: phải thấy câu báo lỗi tiếng Việt và nút Thử lại. Bật server lại, bấm Thử lại thì dữ liệu hiện ra.
- Chạy bản web, mở tab Network của trình duyệt: hai request /books và /categories phải được gửi cùng lúc.
- Bấm vào một cuốn sách: màn Chi tiết hiện vòng quay tải một chút rồi mới ra nội dung.

Bài 7.6: Gọi API danh mục

Yêu cầu: tab Danh mục lấy dữ liệu từ `/categories` và xử lý đủ 3 trạng thái. Cách làm giống Home, chỉ khác là gọi một API.



Hình 11. Danh mục từ API (5 mục; dữ liệu giả chỉ có 4).

Bước 1 — categoryService

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/services/categoryService.ts** TODO

```
export async function getCategories(): Promise<Category[]> {
  // TODO BT7.6a: GET /categories → trả res.data   (cùng khuôn với getBooks)
}
```

Bước 2 — CategoriesScreen

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/screens/CategoriesScreen.tsx** TODO

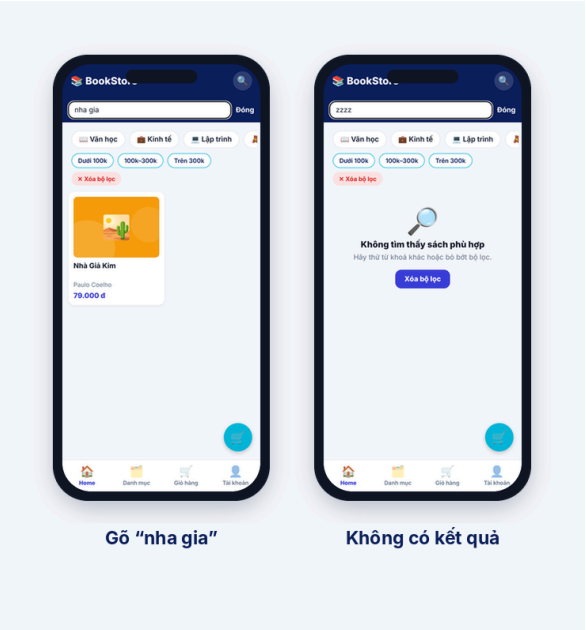
```
// TODO BT7.6b: state data / loading / error + load() gọi getCategories() trong useEffect – giống Home
// TODO BT7.6c: loading → <LoadingView/> ; error → <ErrorView message onRetry={load}/> trước khi vẽ FlatList
const data: Category[] = MOCK_CATEGORIES;   // ← thay bằng state
```

Cách tự kiểm tra:

- Thấy đủ 5 danh mục (bản dữ liệu giả chỉ có 4).
- Thêm một danh mục vào db.json rồi reload app: danh mục mới phải xuất hiện.
- Tắt json-server thì thấy ErrorView; bật lại rồi bấm Thử lại thì có dữ liệu.

Bài 7.7: Tìm kiếm có debounce, không phân biệt dấu

Yêu cầu: bấm 🔍 ở Header rồi gõ từ khoá. Danh sách chỉ lọc sau khi bạn ngừng gõ 400 ms. Gõ “nha gia” phải ra “Nhà Giả Kim”. Không có kết quả thì hiện EmptyState kèm nút Xóa bộ lọc.



Hình 12. Gõ từ khoá → lọc; không có kết quả → EmptyState; nút "✕ Xóa bộ lọc".

Lý thuyết nhanh — debounce

```
// Mỗi lần value đổi: đặt hẹn giờ mới; hàm dọn dẹp HUỶ hẹn giờ cũ → chỉ lần gõ cuối cùng được áp dụng
useEffect(() => {
  const t = setTimeout(() => setDebounced(value), delay);
  return () => clearTimeout(t);
}, [value, delay]);
```

Bước 1 — Viết hook useDebounce

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/hooks/useDebounce.ts** TODO

```
export function useDebounce<T>(value: T, delay = 400): T {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    // TODO BT7.7b: setTimeout(() => setDebounced(value), delay) và return hàm dọn dẹp clearTimeout
    setDebounced(value);   // ← hiện chưa có debounce
  }, [value, delay]);
  return debounced;
}
```

Bước 2 — Lọc theo từ khoá trong filterBooks

A. CODE CÓ SẴN — hàm normalize bỏ dấu tiếng Việt

📄 **src/utils/filters.ts**

```
export const normalize = (s: string) =>
  s.normalize("NFD").replace(/[̀-ͯ]/g, "").replace(/đ/g, "d").replace(/Đ/g, "D").toLowerCase().trim();
```

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/utils/filters.ts** TODO

```
export function filterBooks(books: Book[], f: BookFilter): Book[] {
  // TODO BT7.7c: lọc theo từ khoá – normalize(b.title) hoặc normalize(b.author)
.includes(normalize(f.keyword));
  //              từ khoá rỗng = giữ tất cả
  return books;
}
```

Bước 3 — HomeScreen

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/screens/HomeScreen.tsx** TODO

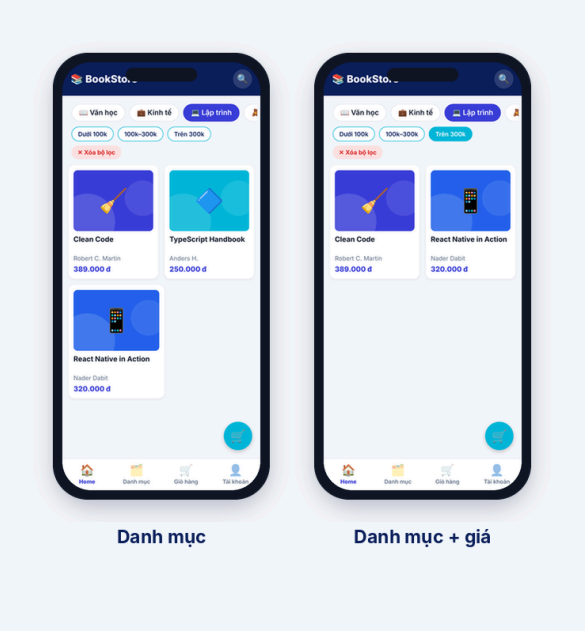
```
// TODO BT7.7a: const debouncedKeyword = useDebounce(keyword, 400);
// TODO BT7.7c: const visibleBooks = useMemo(
//     () => filterBooks(books, { keyword: debouncedKeyword, categoryId, priceRange }),
//     [books, debouncedKeyword, categoryId, priceRange]);
// TODO BT7.7d: ListEmptyComponent = <EmptyState emoji="🔎" title="Không tìm thấy sách phù hợp"
//     message="Hãy thử từ khoá khác hoặc bỏ bớt bộ lọc." actionLabel="Xóa bộ lọc"
onAction={clearFilters} />
// TODO BT7.7e: nút "✕ Xóa bộ lọc" chỉ hiện khi đang có bộ lọc:  hasFilter = keyword !== "" ||
categoryId !== null || ...
```

Cách tự kiểm tra:

- Gõ “DAC NHAN” (không dấu, viết hoa) vẫn ra “Đắc Nhân Tâm”.
- Gõ nhanh liền 10 ký tự: danh sách chỉ lọc một lần, sau khi bạn dừng tay.
- Gõ từ khoá không có trong dữ liệu thì ra EmptyState; bấm Xóa bộ lọc thì quay về đủ 12 sách.

Bài 7.8: Lọc kết hợp và liên kết tab Danh mục

Yêu cầu: bấm một danh mục ở tab Danh mục thì app chuyển về Home và chip danh mục đó được chọn. Danh mục và từ khoá lọc cùng lúc (điều kiện AND).



Hình 13. Danh mục ⟶ Home đã lọc; kết hợp từ khoá + danh mục.

Lý thuyết nhanh

```
navigation.navigate("Home", { categoryId: 3 });        // gửi sang tab Home (tab khác cũng nhận
params)
navigation.setParams({ categoryId: undefined });       // Home xoá params sau khi dùng → không áp
dụng lại
```

Bước 1 — Thêm điều kiện danh mục

B. VIỆC CẦN LÀM — tự code thêm

📄 **src/utils/filters.ts** TODO

```
return books.filter((b) =>
  // TODO BT7.7c: khớp từ khoá (đã làm ở Bài 7.7)
  // TODO BT7.8a: && (f.categoryId === null || b.categoryId === f.categoryId)
  true);
```

Bước 2 — Home nhận categoryId, Danh mục gửi categoryId

B. VIỆC CẦN LÀM — tự code thêm

📄 **HomeScreen.tsx · CategoriesScreen.tsx** TODO

```
// HomeScreen — TODO BT7.8b:
//   useEffect(() => {
//     const id = route.params?.categoryId;
//     if (id !== undefined) { setCategoryId(id); navigation.setParams({ categoryId: undefined });
}
//   }, [route.params?.categoryId, navigation]);

// CategoriesScreen — TODO BT7.8c:
<Pressable style={styles.row} onPress={() => navigation.navigate("Home", { categoryId: item.id
})}>
```

Cách tự kiểm tra:

- Chọn “Lập trình” ở tab Danh mục: về Home, chip “Lập trình” sáng lên. Gõ thêm “react” thì chỉ còn React Native in Action.
- Bấm Xóa bộ lọc thì về đủ 12 sách. Quay lại tab Danh mục chọn mục khác vẫn lọc đúng (nhờ đã xoá params bằng setParams).

Thử thách (cộng điểm)

Mã	Yêu cầu
BT7.C1	Wishlist: viết toggleWishlist trong wishlistSlice, thêm nút ♡/♥ ở BookDetail dùng useWishlist(). Giải thích vì sao nếu sau này đổi thư viện state thì phần giao diện không phải sửa.
BT7.C2	Lọc theo khoảng giá: viết inRange, thêm điều kiện giá vào filterBooks, làm PriceFilter (3 nút, bấm lần nữa để bỏ chọn). Phải dùng chung được với danh mục và từ khoá.

3. Câu hỏi bổ sung

Mỗi câu trả lời 2 đến 4 câu, riêng câu 5 viết thêm đoạn code ngắn. Ghi vào file TraLoi.md đặt trong thư mục project.

1. **Cấu trúc điều hướng.** Vì sao BookDetail được đăng ký ở Stack mà không đặt trong Tab? Nếu đặt vào Tab thì màn hình sẽ khác đi thế nào?

2. **Redux.** Trong reducer viết state.items.push(...) mà sao state gốc không bị sửa trực tiếp? Thử bỏ useMemo trong useCart: kết quả hiển thị có đổi không, và có hậu quả gì?
3. **Promise.all.** Nếu getCategories() lỗi còn getBooks() thành công thì Home hiện gì? Tìm cách để vẫn hiện được danh sách sách (gợi ý: Promise.allSettled) và nói rõ cách đó phải đánh đổi gì.
4. **Debounce.** Vì sao không đặt setTimeout thẳng trong onChangeText? Hàm dọn dẹp (return () => clearTimeout) giúp tránh lỗi gì?
5. **Code bổ sung.** Thêm tuỳ chọn sắp xếp "Giá tăng dần / giảm dần" cho Home. Chỉ cần thêm một state và sửa một chỗ trong visibleBooks. Chụp màn hình kết quả đính kèm.

4. Lỗi thường gặp

Triệu chứng	Nguyên nhân	Cách xử lý
“Network Error” trên điện thoại thật	localhost trên điện thoại không phải máy tính	Đặt EXPO_PUBLIC_API_URL=http://<IP>:3000; cùng Wi-Fi; tường lửa cho phép cổng 3000.
Android emulator không gọi được API	Emulator dùng 10.0.2.2 thay cho localhost	config.ts đã xử lý; kiểm tra json-server chạy với --host 0.0.0.0.
Gõ tìm kiếm lọc ngay không trễ	useDebounce chưa viết (trả value ngay)	Hoàn thành BT7.7b.
Home hiện “Màn Home đang dùng dữ liệu GIẢ”	Chưa xoá TodoBox / chưa nối API	Làm BT7.5b, xoá khung.
Badge không cập nhật	Chưa truyền badge trong CustomTabBar, hoặc totalQuantity vẫn đang trả về 0	Hoàn thành BT7.3b và BT7.3e.
Chọn danh mục rồi quay lại Home vẫn bị lọc	Không xoá params sau khi áp dụng	navigation.setParams({ categoryId: undefined }).
Màn trắng, lỗi “Cannot read property of undefined”	Dùng book.title khi book còn null	Kiểm tra loading/error trước khi render (BT7.5e).
TypeScript báo thiếu tham số khi navigate	Sai tên màn hoặc thiếu params trong types.ts	Đối chiếu RootStackParamList / TabParamList.

5. Danh sách kiểm tra nộp bài

- ☐ Xoá hết thẻ <TodoBox /> và không còn dòng nào chứa “TODO BT7” (trừ thử thách bạn bỏ qua).
- ☐ npx tsc --noEmit không có lỗi.
- ☐ App chạy với json-server: Home, Chi tiết, Giỏ, Thanh toán, Danh mục, Tài khoản đều đúng như UI mô tả.
- ☐ Đã thử tắt json-server: thấy lỗi tiếng Việt và nút Thử lại hoạt động.
- ☐ Chụp 6 ảnh: Home, tìm kiếm, không có kết quả, Chi tiết, Giỏ có badge, Tài khoản đã đăng nhập.
- ☐ Có TraLoi.md với 5 câu hỏi bổ sung.
- ☐ Nén thư mục dự án (KHÔNG gồm node_modules) thành MSSV_HoTen_Buoi7.zip và nộp lên hệ thống.

Phụ lục dành cho giảng viên — đáp án

Đáp án đi theo từng mã TODO trong Starter. Bản chạy hoàn chỉnh là BookStore_Buoi7_Full.zip (đã chạy thử tự động 18 kịch bản trên bản web, tsc không lỗi, chưa thử trên máy thật).

BT7.1 — Điều hướng

📄 **BT7.1**

```
// BT7.1a  CustomTabBar.tsx
const EMOJI: Record<string, string> = { Home: "🏠", Categories: "🗂️", Cart: "🛒", Account: "👤"
};
const items: TabItem[] = state.routes.map((route) => ({
  key: route.key,
  label: descriptors[route.key].options.title ?? route.name,
  emoji: EMOJI[route.name] ?? "•",
  badge: route.name === "Cart" ? totalQuantity : undefined,   // ← BT7.3e
}));
const onPress = (key: string) => {
  const route = state.routes.find((r) => r.key === key);
  if (!route) return;
  const isFocused = state.routes[state.index].key === key;
  const event = navigation.emit({ type: "tabPress", target: route.key, canPreventDefault: true });
  if (!isFocused && !event.defaultPrevented) navigation.navigate(route.name, route.params);
};
return <TabBar items={items} activeKey={state.routes[state.index].key} onPress={onPress} />;

// BT7.1b  TabNavigator.tsx
<Tab.Navigator screenOptions={{ headerShown: false }} tabBar={(props) => <CustomTabBar {...props}
/>}>

// BT7.1c  RootNavigator.tsx
<Stack.Screen name="BookDetail" component={BookDetailScreen} options={{ title: "Chi tiết sách" }}
/>
```

BT7.2 — Tham số

📄 **BT7.2**

```
// BT7.2a HomeScreen
onPressBook={(b) => navigation.navigate("BookDetail", { bookId: b.id })}
// BT7.2b CartScreen
onPress={() => navigation.navigate("Checkout", { total: totalPrice })}
```

BT7.3 — Giỏ hàng

📄 **BT7.3**

```
// BT7.3a cartSlice.ts
addItem(state, action: PayloadAction<Omit<CartItem, "quantity">>) {
  const found = state.items.find((i) => i.bookId === action.payload.bookId);
  if (found) found.quantity = clamp(found.quantity + 1, found.stock);
  else state.items.push({ ...action.payload, quantity: 1 });
},
removeItem(state, action: PayloadAction<number>) {
  state.items = state.items.filter((i) => i.bookId !== action.payload);
},
setQuantity(state, action: PayloadAction<{ bookId: number; quantity: number }>) {
  const found = state.items.find((i) => i.bookId === action.payload.bookId);
  if (found) found.quantity = clamp(action.payload.quantity, found.stock);
},
clearCart(state) { state.items = []; },

// BT7.3b hooks.ts
const totalQuantity = useMemo(() => items.reduce((n, i) => n + i.quantity, 0), [items]);
const totalPrice = useMemo(() => items.reduce((n, i) => n + i.price * i.quantity, 0), [items]);

// BT7.3c BookDetailScreen
onPress={() => add({ bookId: book.id, title: book.title, price: book.price,
                     emoji: book.emoji, color: book.color, stock: book.stock })}
// BT7.3d HomeScreen
const { totalQuantity } = useCart();
<FloatingCartButton count={totalQuantity} onPress={() => navigation.navigate("Cart")} />
// BT7.3f CartScreen
const { items, totalPrice } = useCart();   // và xoá ListEmptyComponent TodoBox → <Text>Giỏ hàng
trống</Text>
```

BT7.4 — Tài khoản

📄 **BT7.4**

```
// BT7.4a authSlice.ts
setCredentials(state, action: PayloadAction<{ user: User; token: string }>) {
  state.user = action.payload.user;
  state.token = action.payload.token;
},
logout(state) { state.user = null; state.token = null; },

// BT7.4b hooks.ts
isLoggedIn: !!token && !!user,

// BT7.4d AccountScreen
onPress={() => signIn({ id: 1, name: "Học viên Demo", email: "demo@bookstore.vn" }, "fake-token")}
onPress={signOut}
```

BT7.5 – BT7.6 — API

📄 **BT7.5 - 7.6**

```
// BT7.5a bookService.ts
export async function getBooks(params?: BookQuery): Promise<Book[]> {
  const res = await api.get<Book[]>("/books", { params });
  return res.data;
}
export async function getBookById(id: number): Promise<Book> {
  const res = await api.get<Book>(`/books/${id}`);
  return res.data;
}
// BT7.6a categoryService.ts
export async function getCategories(): Promise<Category[]> {
  const res = await api.get<Category[]>("/categories");
  return res.data;
}

// BT7.5b/c HomeScreen
```

```
const [books, setBooks] = useState<Book[]>([]);
const [categories, setCategories] = useState<Category[]>([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState<string | null>(null);
const load = useCallback(async () => {
  setLoading(true); setError(null);
  try {
    const [b, c] = await Promise.all([getBooks(), getCategories()]);
    setBooks(b); setCategories(c);
  } catch (e) { setError(getErrorMessage(e)); }
  finally { setLoading(false); }
}, []);
useEffect(() => { load(); }, [load]);
// JSX: loading ? <LoadingView label="Đang tải sách..." /> : error ? <ErrorView message={error}
onRetry={load} /> : <BookGrid ... />

// BT7.5d/e BookDetailScreen — cùng khuôn, phụ thuộc [bookId]
if (loading) return <LoadingView />;
if (error || !book) return <ErrorView message={error ?? "Không tìm thấy sách."} onRetry={load} />;

// BT7.6b/c CategoriesScreen — cùng khuôn với Home nhưng chỉ gọi getCategories()
```

BT7.7 – BT7.8 — Tìm kiếm và lọc

📄 BT7.7 – 7.8

```
// BT7.7b useDebounce.ts
useEffect(() => {
  const t = setTimeout(() => setDebounced(value), delay);
  return () => clearTimeout(t);
}, [value, delay]);

// BT7.7a + 7.7c HomeScreen
const debouncedKeyword = useDebounce(keyword, 400);
const visibleBooks = useMemo(
  () => filterBooks(books, { keyword: debouncedKeyword, categoryId, priceRange }),
  [books, debouncedKeyword, categoryId, priceRange]);

// BT7.7c + 7.8a + C2 filters.ts
export function filterBooks(books: Book[], f: BookFilter): Book[] {
  const k = normalize(f.keyword);
  return books.filter((b) =>
    (k === "" || normalize(b.title).includes(k) || normalize(b.author).includes(k)) &&
    (f.categoryId === null || b.categoryId === f.categoryId) &&
    inRange(b.price, f.priceRange));
}
// C2
const inRange = (p: number, r: PriceRange) =>
  r === null || (r === "lt100" ? p < 100000 : r === "100to300" ? p >= 100000 && p <= 300000 : p >
300000);

// BT7.7d/e HomeScreen
const hasFilter = keyword !== "" || categoryId !== null || priceRange !== null;
const clearFilters = () => { setKeyword(""); setCategoryId(null); setPriceRange(null); };
// ListEmptyComponent={<EmptyState emoji="🔎" title="Không tìm thấy sách phù hợp"
//   message="Hãy thử từ khoá khác hoặc bỏ bớt bộ lọc." actionLabel="Xóa bộ lọc"
onAction={clearFilters} />}
// header: {hasFilter && <Pressable onPress={clearFilters}><Text>✕ Xóa bộ lọc</Text></Pressable>}

// BT7.8b HomeScreen
useEffect(() => {
  const id = route.params?.categoryId;
  if (id !== undefined) { setCategoryId(id); navigation.setParams({ categoryId: undefined }); }
}, [route.params?.categoryId, navigation]);
// BT7.8c CategoriesScreen
onPress={() => navigation.navigate("Home", { categoryId: item.id })}
```

Thử thách

📄 **C1 – C2**

```
// C1 wishlistSlice.ts
toggleWishlist(state, action: PayloadAction<number>) {
  const i = state.ids.indexOf(action.payload);
  if (i >= 0) state.ids.splice(i, 1); else state.ids.push(action.payload);
},
// C2 PriceFilter.tsx
{PRICE_RANGES.map((r) => {
  const active = value === r.key;
  return (
    <Pressable key={r.key} onPress={() => onChange(active ? null : r.key)}
      style={[styles.btn, active && styles.btnActive]}>
      <Text style={[styles.text, active && styles.textActive]}>{r.label}</Text>
    </Pressable>);
})}
```

Gợi ý đáp án câu hỏi bổ sung

1. Đặt ở Stack thì màn Chi tiết phủ lên cả Tab Bar và tự có nút ←. Nếu đặt trong Tab thì nó thành một tab riêng, Tab Bar vẫn hiện và không có nút quay lại.
2. Redux Toolkit dùng Immer: bạn sửa trên một bản nháp, Immer tự tạo ra state mới. Bỏ useMemo thì tổng vẫn đúng, chỉ là bị tính lại ở mỗi lần render, giỏ lớn mới thấy tốn. useMemo chỉ để tối ưu, không quyết định đúng hay sai.
3. Home báo lỗi dù sách đã tải được, vì Promise.all lỗi cả nhóm khi một request lỗi. Dùng Promise.allSettled thì vẫn hiện sách và chỉ thiếu chip danh mục, nhưng phải tự xử lý lỗi cho từng phần.
4. setTimeout đặt thẳng như vậy không bị huỷ khi gõ tiếp, nên mỗi phím vẫn gây một lần lọc (chỉ chậm đi chứ không gộp lại). Hàm dọn dẹp huỷ timer cũ, đồng thời tránh cập nhật state khi component đã bị gỡ (unmount).
5. Thêm state sort ('asc' | 'desc' | null) và sắp xếp bản sao [...filtered].sort((a, b) => a.price - b.price) trong useMemo của visibleBooks.

Rubric gợi ý (10 điểm)

Hạng mục	Điểm	Ghi chú
BT7.1 – 7.2 Điều hướng + tham số	1.5	Tab Bar tự dựng đúng, tham số đúng
BT7.3 – 7.4 Redux	2.0	Giỏ, badge, trạng thái tài khoản
BT7.5 – 7.6 API + 3 trạng thái	2.5	Promise.all, lỗi/thử lại
BT7.7 – 7.8 Tìm kiếm, lọc	2.0	Debounce, không dấu, kết hợp
Câu hỏi bổ sung	1.0	Hiểu bản chất vấn đề
Thử thách C1, C2	1.0	Cộng điểm